HW 7 — Gradient Boosting Regression of Cortical Bone Modulus

BME 5650: Biosolid Mechanics | Model: scikit-learn *GradientBoostingRegressor* | Companion notebook: `HW7_gradient_boosting.ipynb`

1. Problem and approach

The goal is to predict Young's modulus E (GPa) of cortical bone from calcium content Ca (mg/g) and volume fraction V_f, without choosing a physical functional form in advance. I used a gradient boosting regressor (GBR), an ensemble of shallow decision trees fit sequentially, and compared it to the 3-parameter Currey-type power law $E = a \cdot Ca^b \cdot V_f^c$ from class. The data set has 370 rows; one row with a missing V_f was dropped, leaving 369 samples, split 75:25 into training (276) and test (93) sets with a fixed random seed (42). Only `Ca_mg_g` and `Vf` were used as inputs; `E_GPa` is the target.

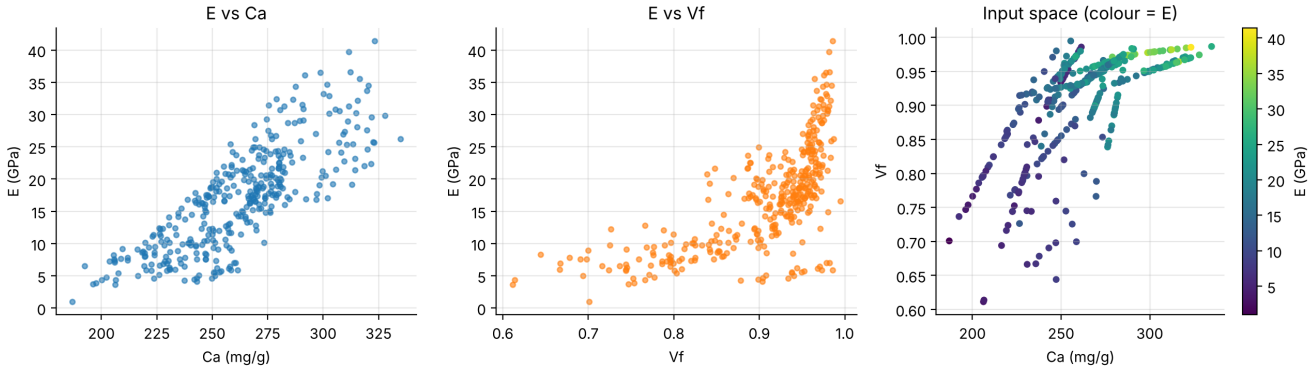


Figure 1. The data. E rises with both Ca (r = 0.81) and Vf (r = 0.68); the two inputs are correlated with each other (r = 0.68). Note the distinct streaks in input space (right), which are separate specimen groups.

2. How gradient boosting works

Gradient boosting builds an additive model $F_M(x) = F_0 + \nu \, \Sigma_m \, h_m(x)$, where each h_m is a small regression tree. For squared-error loss the algorithm is:

- Start with a constant: $F_0(x) = mean(y)$.
- For $m = 1 \ldots M$ (`n_estimators`): compute residuals $r_i = y_i - F_{m-1}(x_i)$ (the negative gradient of the loss); fit a tree of depth ≤ `max_depth` to those residuals, using a random fraction `subsample` of rows and leaves of at least `min_samples_leaf` samples.
- Update $F_m = F_{m-1} + \nu \cdot h_m$, where ν is the `learning_rate` (shrinkage). Each new tree corrects what the previous trees got wrong; a small ν takes smaller steps and needs more trees.

This is the approach shown in the scikit-learn examples *"Gradient Boosting regression"* (deviance vs. boosting iteration, feature importance) and *"Gradient Boosting regularization"* (effect of shrinkage and subsampling), which I followed for the learning-curve and regularization analyses.

3. Functions and methods used

Function / class (library)	Purpose in this analysis	Key settings
pandas.read_csv, DataFrame.dropna	Load CSV; remove the one row missing Vf	subset=['Vf']
sklearn.model_selection.train_test_split	75:25 train/test split	test_size=0.25, random_state=42

Function / class (library)	Purpose in this analysis	Key settings
scipy.optimize.curve_fit	Nonlinear least-squares fit of the Currey power law (baseline)	p0=[1e-5, 2.5, 2.5]; fit on train only
sklearn.ensemble.GradientBoostingRegressor	The model: boosted regression trees	loss='squared_error'; tuned n_estimators, learning_rate, max_depth, min_samples_leaf, subsample
sklearn.model_selection.KFold + GridSearchCV	Choose hyperparameters by 5-fold CV inside the training set (384 combinations)	scoring='neg_root_mean_squared_error', shuffle=True
sklearn.model_selection.cross_val_score	CV RMSE for one-at-a-time parameter sweeps and the lr × n_estimators map	cv=5-fold KFold
sklearn.metrics: mean_squared_error, mean_absolute_error, r2_score	Model quality: RMSE, MAE, R^2 on train and test	RMSE = √MSE
GradientBoostingRegressor.staged_predict	Prediction after each added tree → error vs. number of trees	1000 trees, lr=0.1, depth 3
feature_importances_; sklearn.inspection.permutation_importance	Which input matters most (impurity-based, and ΔR^2 when an input is shuffled on test data)	n_repeats=50, scoring='r2'
sklearn.inspection.PartialDependenceDisplay	Average effect of each input (and ICE curves) to compare with Currey trends	kind='both'
sklearn.preprocessing.StandardScaler + make_pipeline	Test whether input normalization changes the result	also tried log(X)

4. Results

Model	Set	RMSE (GPa)	MAE (GPa)	R^2
Currey power law	train	4.30	3.23	0.705
Currey power law	test	4.50	3.40	0.655
GBR default	train	2.22	1.69	0.921
GBR default	test	4.23	3.08	0.696
GBR tuned	train	3.20	2.39	0.837
GBR tuned	test	4.13	3.03	0.710

Table 1. Model quality. Tuned GBR hyperparameters (best 5-fold CV RMSE = 4.06 GPa): n_estimators = 100, learning_rate = 0.1, max_depth = 2, min_samples_leaf = 10, subsample = 0.7. Currey fit on training data: a = 4.36×10^{-5}, b = 2.35, c = 2.63.

The default GBR over-fits (train R^2 0.92 vs. test 0.70). Tuning toward shallower trees, larger leaves and subsampling reduced the train–test gap and gave the best test performance: RMSE 4.13 GPa and R^2 0.71, versus 4.50 GPa and 0.66 for the Currey power law. The improvement is real but modest: much of the remaining error cannot be explained by Ca and Vf alone, because different specimen groups have different E at the same composition (Figure 1).

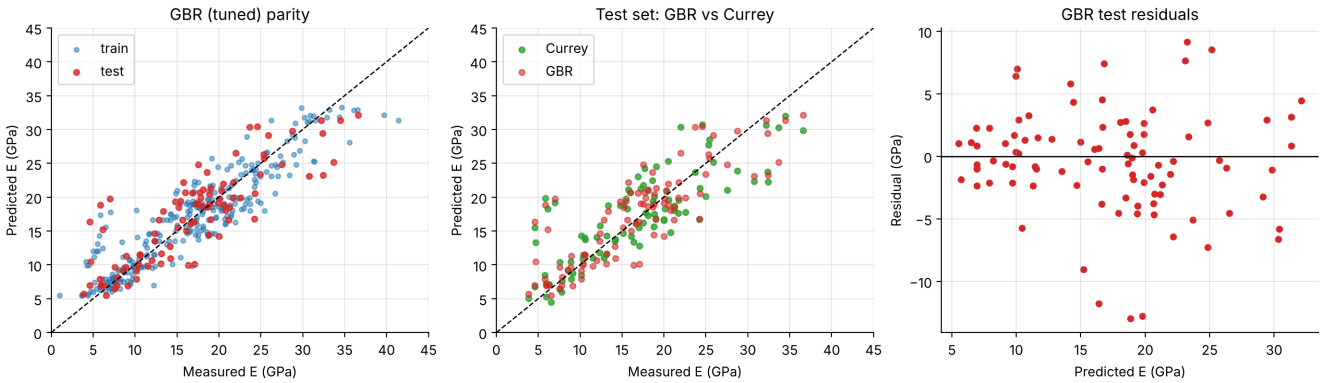


Figure 2. Left: measured vs. predicted E for the tuned GBR. Middle: test-set comparison with Currey. Right: test residuals; scatter is roughly uniform, with the largest errors for the stiffest specimens (E > 30 GPa).

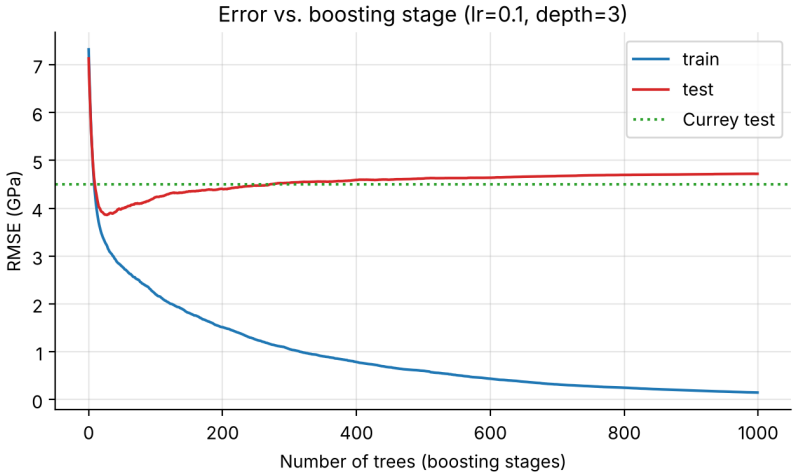


Figure 3. RMSE vs. number of trees (lr = 0.1, depth 3). Training error keeps falling toward zero while test error reaches its minimum around 30 trees and then slowly rises: the classic sign of over-fitting.

Effect of parameter ranges on RMSE and R²

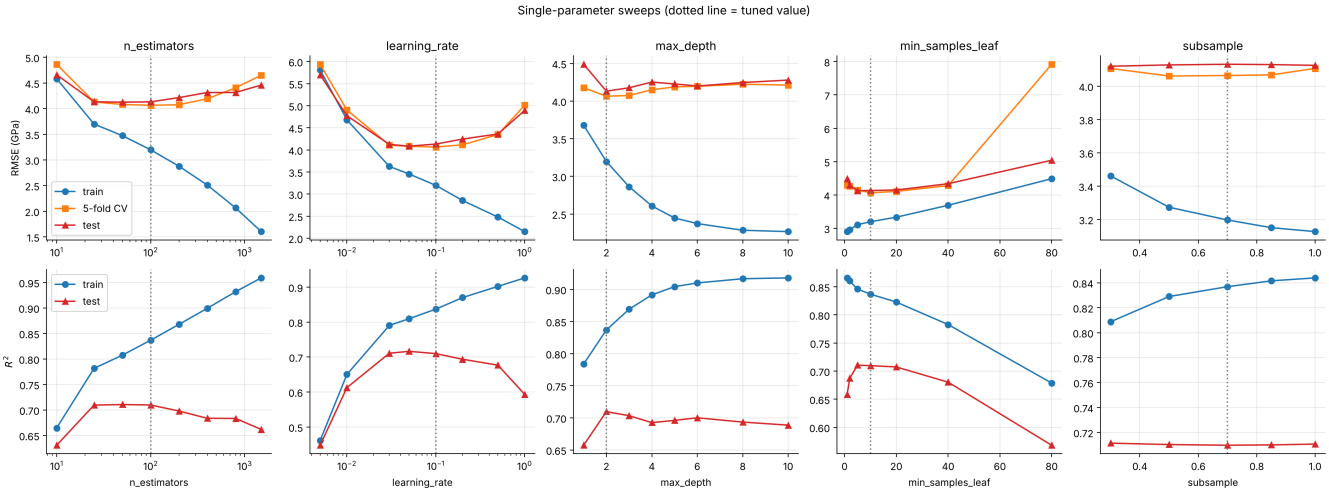


Figure 4. Each hyperparameter varied alone, others at tuned values. Top: RMSE (train, 5-fold CV, test). Bottom: R².

Parameter (range tried)	Effect
n_estimators (10–1500)	Train RMSE falls steadily (4.6 → 1.6 GPa). Test RMSE best at 25–100 trees (4.13), rising to 4.46 at 1500 (test R^2 0.71 → 0.66). Too few trees (10) under-fit (test R^2 0.63).
learning_rate (0.005–1.0)	With 100 trees, lr ≤ 0.01 under-fits badly (test R^2 0.45 at 0.005). Best at 0.03–0.1 (test R^2 ≈ 0.71–0.72). lr = 1.0 over-fits (test R^2 0.59). Smaller lr needs proportionally more trees (Figure 5).
max_depth (1–10)	Depth 1 (stumps: additive model, no Ca×Vf interaction) under-fits (test R^2 0.66). Depth 2–3 is best; deeper trees fit training data better (R^2 → 0.92) without improving test.
min_samples_leaf (1–80)	Small leaves (1–2) over-fit (test R^2 0.66); 5–20 is best; 80 is too coarse and under-fits (test R^2 0.57).
subsample (0.3–1.0)	Little effect on test error here (≈ 4.12–4.13 GPa); lowers train fit slightly (regularizing).

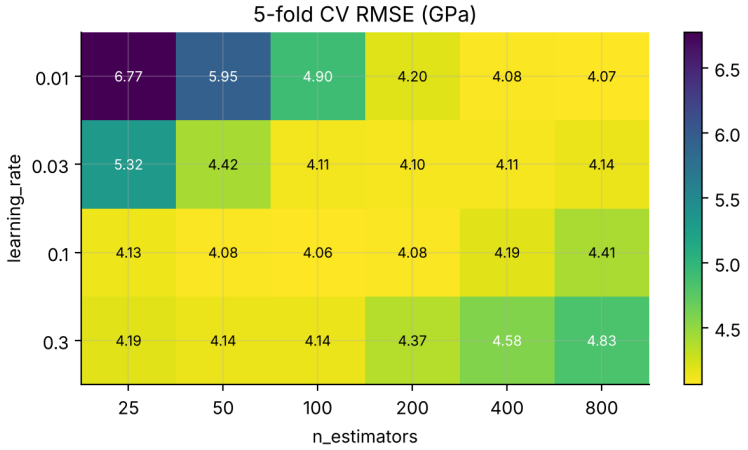


Figure 5. 5-fold CV RMSE over learning rate × number of trees. Good models lie along a diagonal: a smaller learning rate needs more trees.

Interpretation: importance and partial dependence

Calcium is the dominant input: impurity importance 0.67 (Ca) vs. 0.33 (Vf); permutation importance on the test set $\Delta R^2 = 0.70 \pm 0.11$ (Ca) vs. 0.33 ± 0.05 (Vf).

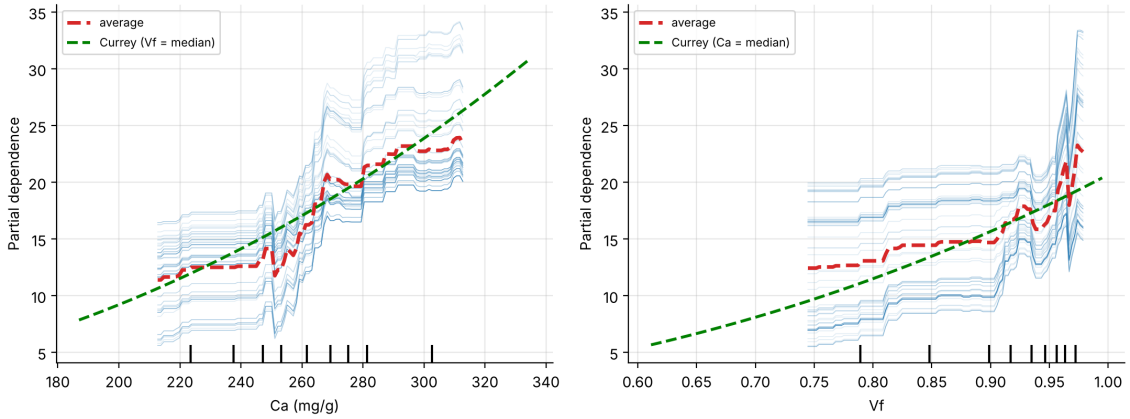


Figure 6. Partial dependence (red, average) and individual conditional expectation curves (blue) for the tuned GBR, with the Currey power law overlaid (green, other input at its median). Both show E rising with Ca and Vf; GBR is a staircase that flattens at the ends of the data.

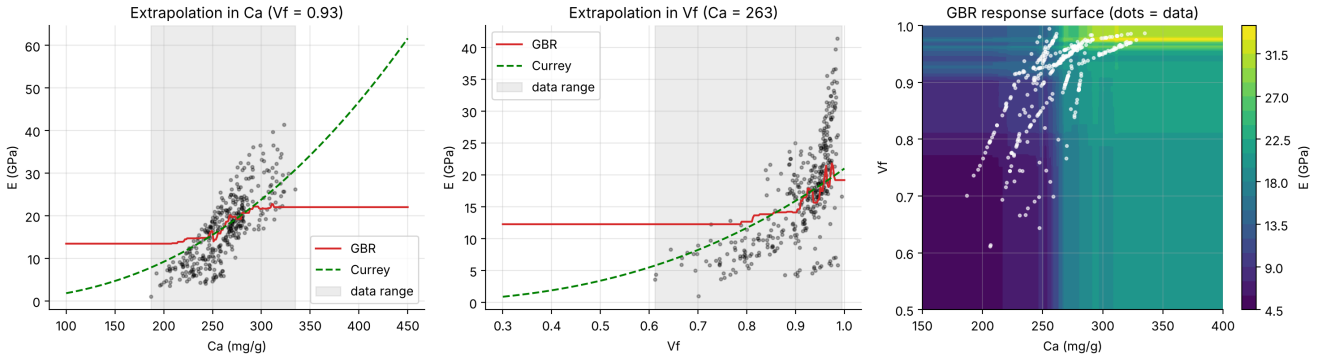


Figure 7. Behaviour outside the data range (grey band). GBR (red) becomes perfectly flat beyond the last split; the Currey law (green) keeps changing as a power law. Right: GBR response surface — rectangular, piecewise-constant regions.

5. Answers to the discussion questions

Are any of the observed trends consistent with the physical model proposed by Currey? Yes. Without being told any functional form, GBR learned that E increases with both Ca and Vf (Figure 6), and the response surface (Figure 7) shows the inputs act jointly — the highest moduli occur only when both mineral content and volume fraction are high — consistent with a multiplicative law $E \propto Ca^{b}V_{f}^{c}$. Depth-1 trees (purely additive, no interaction) performed worse than depth 2–3, which supports a Ca × Vf interaction. Ca being the more important variable matches mineralization as the main stiffness determinant. Differences: GBR's curves are step-like with small local wiggles (fits to noise), and they saturate at the data edges rather than following a smooth power law.

Did you need to normalize the data at all? No. Trees split on thresholds of one variable at a time, so any monotonic rescaling (standardization, min–max, log) gives the same splits. I verified this: raw, StandardScaler and log-transformed inputs gave an identical test RMSE of 4.1317 GPa and predictions differing by 0.0 GPa. This contrasts with kernel ridge or MLP models, where scaling is essential. The only cleaning needed was removing the row with a missing Vf.

What are the weaknesses of this approach? Its strengths?

- **Strengths:** no assumed physical form; captures non-linearity and interactions automatically; no need for scaling; robust to outliers in the inputs; gives feature importances; usually strong accuracy on small tabular data sets; shrinkage, subsampling and leaf size give good control of over-fitting.
- **Weaknesses:** several interacting hyperparameters that must be tuned by cross-validation; over-fits easily (train R^2 up to 0.96 while test R^2 falls); the prediction is piecewise constant, so it is not smooth and has no meaningful derivatives (e.g. $\partial E/\partial Ca$); no physical parameters like Currey's exponents to interpret or compare across studies; no built-in uncertainty estimate (quantile loss would be needed); trains sequentially; and it performs badly outside the data range. Here it gave only a modest gain over a 3-parameter physical model.

How would it work if the dimensionality of your input/output varied? More inputs (many compositional measures — collagen, water, porosity, carbonate, crystallinity, etc.) are handled well: each split chooses the most useful variable, uninformative inputs are largely ignored, and importances help select features. More data would be needed to avoid over-fitting as dimension grows, and highly correlated inputs split importance between them. GradientBoostingRegressor predicts only one output, so several mechanical properties (E, strength, toughness) would need one model per output (MultiOutputRegressor) or a RegressorChain to use correlations between outputs; a physical model would need a new equation derived for each property.

How well is it predicted to work far outside of the data range? Poorly. A tree ensemble cannot extrapolate: beyond the last split threshold the prediction is constant. For example, at Vf = 0.99 GBR

predicts 31.3 GPa for Ca = 335, 400 and 450 mg/g alike, and at low composition (Ca = 150, Vf = 0.6) it predicts 5.5 GPa, the same as the lowest training region (Figure 7). The Currey law continues its trend (e.g. 1.5 GPa at the low end, 54.5 GPa at Ca = 400), which is more physically plausible though also unverified. GBR should only be trusted inside the range of the training data; a physics-based model is preferable for extrapolation.

6. Written summary (< 200 words)

I modeled cortical bone modulus E from calcium content and volume fraction using a gradient boosting regressor, an ensemble of shallow decision trees fit sequentially to residuals. After dropping one incomplete row, I used a 75:25 split and tuned five hyperparameters by 5-fold cross-validation on the training set. The tuned model (100 trees, learning rate 0.1, depth 2) reached test RMSE 4.13 GPa and R^2 0.71, slightly better than Currey's power law (4.50 GPa, 0.66). The default model over-fit (train R^2 0.92); more trees, higher learning rate, deeper trees and smaller leaves all lowered training error while increasing test error. Without assuming any physics, the model recovered Currey's trends: E rises with both inputs, Ca dominates, and the inputs interact. Normalization was unnecessary because tree splits are scale-invariant. The main weaknesses are a step-like, non-smooth response, no interpretable parameters, and flat, unrealistic predictions outside the data range. From the class discussion I learned that flexible machine-learning models fit well within the data but cannot replace physical models for extrapolation, and that unmeasured differences between specimen groups limit any model built from composition alone.

(183 words)